

Sistema Escolar

Parte 3: Repository, Persistencia y Robustez

¿De dónde venimos?

En la Parte 2 transformaste el sistema escolar incorporando **clases abstractas**, **interfaces**, el **patrón Strategy** para calcular promedios y el **patrón MVC** para separar responsabilidades.

Sin embargo, si cerrás el programa **todos los datos se pierden**. Tampoco hay validaciones robustas: podés cargar un legajo duplicado sin que el sistema lo detecte, y los errores se comunican con `Console.WriteLine()` en lugar de mecanismos más apropiados.

Esta tercera parte resuelve exactamente esos problemas, dando como resultado un sistema que respeta prácticas de desarrollo de software limpio y escalable.

Concepto	Para qué lo usamos
Excepciones personalizadas	Comunicar errores de forma precisa y estructurada. El método que falla lanza la excepción; quien lo llama decide cómo manejarla.
Patrón Repository	Desacoplar el acceso a datos del resto del sistema. El Controller no sabe si los datos vienen de memoria, un archivo JSON o una base de datos.
Persistencia con JSON	Guardar y recuperar el estado completo del sistema entre ejecuciones, usando la biblioteca <code>System.Text.Json</code> incluida en .NET.
Menú de consola en <code>Program.cs</code>	Centralizar la configuración inicial: el usuario elige cómo arrancar el sistema antes de que comience la sesión de trabajo.

Paso 1 — Robustez: Excepciones Personalizadas

¿Por qué no alcanza con `Console.WriteLine`?

En la versión actual, cuando algo sale mal (cupó lleno, legajo no encontrado), el sistema imprime un mensaje por consola y sigue ejecutando. Esto tiene un problema grave: **el código que llamó a ese método no sabe que algo falló**. No puede reaccionar de forma diferente, no puede intentar otra acción, no puede decidir qué mostrarle al usuario.

La solución es usar **excepciones**. Una excepción es una señal estructurada que el método lanza cuando no puede completar su tarea. El código que lo invoca puede capturarla (`try/catch`) y manejarla de la forma apropiada. Esto devuelve el control al lugar correcto.

Principio de diseño: separación del manejo de errores

Un método que no puede completar su tarea debe **lanzar una excepción**, no imprimir un mensaje. Quien invoca al método decide cómo manejarlo.

Ventaja concreta: si mañana reemplazás la interfaz de consola por una interfaz gráfica, los `Console.WriteLine()` de error dejan de funcionar. Con excepciones, solo cambiás cómo las capturás, no cómo las generás.

Crear las excepciones personalizadas

Creá una carpeta **Excepciones/** dentro del proyecto y agregá estos archivos. Cada excepción describe un problema específico del dominio, con un mensaje claro:

Excepciones/LegajoYaExisteException.cs

```
public class LegajoYaExisteException : Exception
{
    public int Legajo { get; }

    public LegajoYaExisteException(int legajo)
        : base($"Ya existe un alumno con legajo {legajo}.")
    {
        Legajo = legajo;
    }
}
```

Excepciones/SinCupoException.cs

```
public class SinCupoException : Exception
{
    public SinCupoException(string nombreMateria)
        : base($"La materia '{nombreMateria}' no tiene cupo disponible.")
    { }
}
```

Excepciones/EntidadNoEncontradaException.cs

```
public class EntidadNoEncontradaException : Exception
{
    public EntidadNoEncontradaException(string tipo, string identificador)
        : base($"{tipo} con identificador '{identificador}' no encontrado/a.")
    { }
}
```

Actualizar Materia para lanzar excepciones

La clase **Materia** ya no imprime por consola cuando no hay cupo. En cambio, lanza una excepción. El Controller es quien decide qué mensaje mostrarle al usuario:

Materia.cs — método Inscribir actualizado

```
// Implementación de IInscribible
public void Inscribir(Persona persona)
{
    // Si no hay cupo, lanzamos una excepción específica del dominio
    if (Inscriptos.Count >= Cupo)
        throw new SinCupoException(Nombre);

    // Evitamos inscripción duplicada
    if (Inscriptos.Contains(persona))
        throw new InvalidOperationException(
            $"{persona.Nombre} ya está inscripto en {Nombre}.");

    Inscriptos.Add(persona);
}
```

Ejercicio 1.A: Crear y usar las excepciones

1. Creá la carpeta Excepciones/ y los tres archivos de excepciones personalizadas.
2. Actualizá `Materia.Inscribir()` para que lance `SinCupoException` en lugar de imprimir.
3. En el Controller (método `InscribirAlumno`), envolvé la llamada con `try/catch` y mostrá el mensaje de la excepción al usuario.
4. Verificá que al intentar inscribir más alumnos que el cupo, el mensaje aparece correctamente.

Ejercicio 1.B — Reflexión

¿Por qué es mejor que `Materia` lance una excepción en vez de imprimir directamente?

Pensá qué pasaría si quisieras usar este sistema con una interfaz gráfica: ¿funcionaría el `Console.WriteLine()` de error?

Anotá tu respuesta en un comentario dentro de `Materia.cs`.

Paso 2 — Menú de Consola en Program.cs

¿Para qué sirve el menú inicial?

Hasta ahora Program.cs simplemente crea el MVC y llama a **controller.Ejecutar()**. En la Parte 3 necesitamos hacer algo antes de arrancar: preguntarle al usuario si quiere trabajar con datos en memoria o con datos persistidos en disco.

Este tipo de configuración inicial pertenece naturalmente a Program.cs, el punto de entrada del sistema. No es lógica de negocio ni presentación: es **ensamblado del sistema**, que es exactamente la responsabilidad de Program.cs en un esquema MVC limpio.

Principio de diseño: un solo punto de decisión

Program.cs es el único lugar que sabe qué implementación concreta de repositorio se usa. El Controller solo conoce interfaces (IRepositorioAlumnos, IRepositorioMaterias).

Si mañana agregás una implementación que guarda en SQL Server, solo cambiás Program.cs. El resto del sistema no se toca.

Código del menú de inicio

Program.cs — menú de configuración inicial

```
using System;

Console.OutputEncoding = System.Text.Encoding.UTF8;

Console.WriteLine(" ");
Console.WriteLine(" SISTEMA ESCOLAR v3.0 ");
Console.WriteLine(" ");
Console.WriteLine();
Console.WriteLine("¿Cómo desea iniciar el sistema?");
Console.WriteLine(" 1. Datos en memoria (sin persistencia)");
Console.WriteLine(" 2. Datos persistidos en disco (JSON)");
Console.Write("Opción: ");

string opcionInicio = Console.ReadLine() ?? "1";

// Elegimos los repositorios según la preferencia del usuario.
// El Controller nunca sabrá cuál de los dos se está usando.
IRepositorioAlumnos repoAlumnos;
IRepositorioMaterias repoMaterias;

if (opcionInicio == "2")
{
    repoAlumnos = new RepositorioAlumnosJson("alumnos.json");
    repoMaterias = new RepositorioMateriasJson("materias.json", repoAlumnos);
    Console.WriteLine("→ Cargando datos desde disco...");
}
else
{
    repoAlumnos = new RepositorioAlumnosMemoria();
    repoMaterias = new RepositorioMateriasMemoria();
    repoAlumnos.Agregar(new Alumno("Ana", "ana@mail.com", 1001));
    repoAlumnos.Agregar(new Alumno("Bruno", "bruno@mail.com", 1002));
    repoMaterias.Agregar(new Materia("MAT101", "Matematica I", 3));
    Console.WriteLine("→ Sistema iniciado con datos de prueba.");
}

// Ensamblar el MVC con los repositorios elegidos
var model = new EscolarModel(repoAlumnos, repoMaterias);
var view = new EscolarView();
var controller = new EscolarController(model, view);

controller.Ejecutar();
```

Nota importante

Las clases `RepositoryAlumnosJson` y `RepositoryMateriasJson` todavía no existen. Completá este paso dejando comentada la opción 2 en `Program.cs`. Las implementarás en el Paso 4. La opción 1 (memoria) debe funcionar correctamente desde ahora.

Ejercicio 2.A: Implementar el menú de inicio

1. Reemplazá tu `Program.cs` actual por el código de arriba (dejá comentada la rama del "2" por ahora).
2. Verificá que la opción 1 (memoria) funciona igual que antes.
3. Observá que `Program.cs` no contiene ninguna lógica de negocio: solo configura y ensambla el sistema.

Ejercicio 2.B — Reflexión

¿Qué pasaría si no tuvieras este menú de configuración y la elección de repositorio estuviera dentro del Controller?

¿Qué parte del sistema habría que modificar si querés agregar una tercera opción (por ejemplo, guardar en XML)?

Paso 3 — Patrón de Diseño: Repository

¿Qué es el patrón Repository?

El patrón Repository actúa como un intermediario entre la lógica de negocio y el mecanismo de almacenamiento. Desde el punto de vista del Controller o del Model, un repositorio es simplemente **una colección de objetos** con operaciones básicas: agregar, buscar, listar y guardar (un CRUD digamos).

No importa si esa colección vive en memoria, en un archivo o en una base de datos. Eso es un detalle de implementación que el repositorio oculta detrás de una interfaz.

Analogía: la biblioteca

Cuando pedís un libro en una biblioteca, le decís el título al bibliotecario. No sabés si está en el estante A, en el depósito, o en otra sucursal. Eso es problema del bibliotecario, no tuyo.

En el código: el Controller pide un Alumno por legajo. No sabe si viene de memoria o de un archivo JSON. Solo pide y recibe.

Principio D de SOLID: Inversión de Dependencias

En la versión anterior, EscolarModel guardaba internamente sus propias listas. Eso hace que EscolarModel dependa de los detalles concretos del almacenamiento. Con el patrón Repository, el Model depende solo de **interfaces abstractas**. Las implementaciones concretas se inyectan desde afuera.

Esto es el **principio D de SOLID** (Dependency Inversion): "Los módulos de alto nivel no deben depender de los de bajo nivel. Ambos deben depender de abstracciones."

Código: Las interfaces del Repository

Creá una carpeta **Repositorios/** y agregá estos tres archivos de interfaces:

Repositorios/IRepository.cs

```
// Contrato genérico para cualquier repositorio del sistema.
// T es el tipo de entidad que maneja (Alumno, Materia, etc.)
public interface IRepository<T>
{
    void    Agregar(T entidad);
    T?     BuscarPorId(string id);
    List<T> ObtenerTodos();
    void    Guardar(); // Persiste los cambios. Puede ser un no-op en memoria.
}
```

Repositorios/IRepositoryAlumnos.cs

```
public interface IRepositoryAlumnos : IRepository<Alumno>
{
    // Búsqueda específica del dominio escolar
    Alumno? BuscarPorLegajo(int legajo);
}
```

Repositorios/IRepositoryMaterias.cs

```
public interface IRepositoryMaterias : IRepository<Materia>
{
    Materia? BuscarPorCodigo(string codigo);
}
```

Código: Implementaciones en memoria

Las implementaciones en memoria son simples: usan listas internas. La novedad es que ahora lanzan excepciones en lugar de ignorar errores silenciosamente:

Repositorios/RepositorioAlumnosMemoria.cs

```
public class RepositorioAlumnosMemoria : IRepositoryAlumnos
{
    private readonly List<Alumno> _alumnos = new();

    public void Agregar(Alumno alumno)
    {
        if (BuscarPorLegajo(alumno.Legajo) != null)
            throw new LegajoYaExisteException(alumno.Legajo);
        _alumnos.Add(alumno);
    }

    public Alumno? BuscarPorId(string id) =>
        _alumnos.FirstOrDefault(a => a.Legajo.ToString() == id);

    public Alumno? BuscarPorLegajo(int legajo) =>
        _alumnos.FirstOrDefault(a => a.Legajo == legajo);

    public List<Alumno> ObtenerTodos() => new(_alumnos);

    // En memoria no necesitamos persistir nada
    public void Guardar() { }
}
```

Repositorios/RepositorioMateriasMemoria.cs

```
public class RepositorioMateriasMemoria : IRepositoryMaterias
{
    private readonly List<Materia> _materias = new();

    public void Agregar(Materia m) => _materias.Add(m);
    public Materia? BuscarPorId(string id) => BuscarPorCodigo(id);

    public Materia? BuscarPorCodigo(string codigo) =>
        _materias.FirstOrDefault(m => m.Codigo == codigo);

    public List<Materia> ObtenerTodos() => new(_materias);
    public void Guardar() { }
}
```

Actualizar EscolarModel para usar los repositorios

EscolarModel ya no mantiene sus propias listas. Recibe los repositorios por constructor (**inyección de dependencias**) y los usa. El Model no sabe si los repositorios son en memoria o JSON:

EscolarModel.cs — versión con Repository

```
public class EscolarModel
{
    private readonly IRepositoryAlumnos _repoAlumnos;
    private readonly IRepositoryMaterias _repoMaterias;

    // Los repositorios se inyectan desde Program.cs
    public EscolarModel(IRepositoryAlumnos repoAlumnos,
                       IRepositoryMaterias repoMaterias)
    {
        _repoAlumnos = repoAlumnos;
    }
}
```

```

        _repoMaterias = repoMaterias;
    }

    public void    AgregarAlumno(Alumno a)    => _repoAlumnos.Agregar(a);
    public void    AgregarMateria(Materia m) => _repoMaterias.Agregar(m);

    public List<Alumno> ObtenerAlumnos() => _repoAlumnos.ObtenerTodos();
    public List<Materia> ObtenerMaterias() => _repoMaterias.ObtenerTodos();

    public Alumno? BuscarAlumno(int legajo)    =>
        _repoAlumnos.BuscarPorLegajo(legajo);
    public Materia? BuscarMateria(string codigo) =>
        _repoMaterias.BuscarPorCodigo(codigo);

    // Llama a Guardar() en todos los repositorios.
    // En memoria no hace nada; en JSON guarda los archivos.
    public void GuardarTodo()
    {
        _repoAlumnos.Guardar();
        _repoMaterias.Guardar();
    }
}

```

Ejercicio 3.A: Implementar el patrón Repository

1. Creá la carpeta Repositorios/ con los archivos de interfaces.
2. Implementá RepositorioAlumnosMemoria y RepositorioMateriasMemoria.
3. Actualizá EscolarModel para que reciba y use los repositorios por constructor.
4. Actualizá Program.cs para inyectar los repositorios en EscolarModel.
5. Verificá que el sistema sigue funcionando igual. El Controller no debería necesitar cambios.

Ejercicio 3.B — Reflexión

- ¿Qué ventaja concreta tiene que EscolarController no sepa qué implementación de repositorio se está usando?
- ¿Qué tendrías que modificar si agregaras una implementación que guarda en SQL Server?
- ¿Cuál es la diferencia entre IRepositoryo<T> e IRepositoryoAlumnos?

Paso 4 — Persistencia con JSON

¿Por qué JSON?

JSON (JavaScript Object Notation) es un formato de texto simple, legible por humanos, y el estándar más usado para intercambiar datos en aplicaciones modernas. .NET incluye la biblioteca **System.Text.Json** que convierte objetos C# a JSON y viceversa con muy poco código.

Usaremos dos archivos: **alumnos.json** y **materias.json**. Cada repositorio JSON conoce su propio archivo y es responsable de leerlo al iniciar y escribirlo al guardar.

El problema de serializar clases abstractas

C# no puede serializar directamente objetos de clases abstractas como *Persona*, porque al deserializar no sabe qué tipo concreto crear.

La solución es usar **DTOs (Data Transfer Objects)**: clases simples y concretas que representan exactamente los datos que queremos guardar. Convertimos *Alumno* → *AlumnoDto* para guardar, y *AlumnoDto* → *Alumno* para cargar. Los DTOs no tienen lógica: solo tienen propiedades.

DTOs: clases de datos para serialización

Creá una carpeta **DTOs/** con estos archivos. Observá que son clases concretas, sin herencia ni lógica:

DTOs/AlumnoDto.cs

```
// DTO: solo datos, sin lógica, para serialización/deserialización.
public class AlumnoDto
{
    public string      Nombre      { get; set; } = "";
    public string      Email       { get; set; } = "";
    public int         Legajo       { get; set; }
    public List<double> Notas       { get; set; } = new();
    // Guardamos el nombre de la estrategia como string para reconstruirla
    public string      Estrategia { get; set; } = "Simple";
}
```

DTOs/MateriaDto.cs

```
public class MateriaDto
{
    public string      Codigo { get; set; } = "";
    public string      Nombre { get; set; } = "";
    public int         Cupo   { get; set; }
    // Solo guardamos los legajos, no los objetos Alumno completos.
    public List<int> LegajosInscriptos { get; set; } = new();
}
```

Ajustes necesarios en la clase Alumno

Para que el repositorio JSON pueda leer y escribir las notas y la estrategia, la clase *Alumno* necesita tres adiciones. Son métodos de soporte para la persistencia, no lógica de negocio:

Alumno.cs — métodos para persistencia (agregar a la clase)

```
// Permite que el repo JSON lea las notas para guardarlas en disco
public List<double> ObtenerNotas() => new List<double>(_notas);

// Permite que el repo JSON recargue notas al iniciar
public void CargarNotas(IEnumerable<double> notas)
{
    _notas.Clear();
    foreach (var n in notas) _notas.Add(n);
}

// Devuelve el nombre de la estrategia actual para guardarlo como string.
public string NombreEstrategia => _estrategia.GetType().Name;
```

EstrategiaFactory: reconstruir estrategias desde texto

Al cargar desde JSON, tenemos el nombre de la estrategia como texto. Necesitamos convertirlo en el objeto correcto. Para eso creamos una **Factory** simple:

EstrategiaFactory.cs

```
public static class EstrategiaFactory
{
    public static IEstrategiaPromedio Crear(string nombre) => nombre switch
    {
        nameof(PromedioPonderado) => new PromedioPonderado(),
        nameof(PromedioSinExtremos) => new PromedioSinExtremos(),
        nameof(PromedioAprobacion) => new PromedioAprobacion(),
        _ => new PromedioSimple(), // default
    };
}
```

Repositorio de Alumnos con JSON

Este repositorio implementa la misma interfaz que la versión en memoria, pero carga los datos de un archivo al iniciar y los guarda al llamar a **Guardar()**. Para el Controller, esto es completamente transparente.

Repositorios/RepositorioAlumnosJson.cs

```
using System.Text.Json;

public class RepositorioAlumnosJson : IRepositoryAlumnos
{
    private readonly string _path;
    private readonly List<Alumno> _alumnos;

    public RepositorioAlumnosJson(string path)
    {
        _path = path;
        _alumnos = Cargar();
    }

    // — Carga desde disco —————
    private List<Alumno> Cargar()
    {
        if (!File.Exists(_path)) return new List<Alumno>();
        string json = File.ReadAllText(_path);
        var dtos = JsonSerializer.Deserialize<List<AlumnoDto>>(json) ?? new();
        return dtos.Select(dto =>
        {
            var alumno = new Alumno(dto.Nombre, dto.Email, dto.Legajo,
                EstrategiaFactory.Crear(dto.Estrategia));
        });
    }
}
```

```

        alumno.CargarNotas(dto.Notas);
        return alumno;
    }).ToList();
}

// — Guarda a disco —————
public void Guardar()
{
    var dtos = _alumnos.Select(a => new AlumnoDto
    {
        Nombre = a.Nombre, Email = a.Email, Legajo = a.Legajo,
        Notas = a.ObtenerNotas(), Estrategia = a.NombreEstrategia
    }).ToList();
    string json = JsonSerializer.Serialize(dtos,
        new JsonSerializerOptions { WriteIndented = true });
    File.WriteAllText(_path, json);
}

// — Implementación de la interfaz —————
public void Agregar(Alumno a)
{
    if (BuscarPorLegajo(a.Legajo) != null)
        throw new LegajoYaExisteException(a.Legajo);
    _alumnos.Add(a);
}

public Alumno? BuscarPorId(string id) =>
    _alumnos.FirstOrDefault(a => a.Legajo.ToString() == id);
public Alumno? BuscarPorLegajo(int l) =>
    _alumnos.FirstOrDefault(a => a.Legajo == l);
public List<Alumno> ObtenerTodos() => new(_alumnos);
}

```

Repositorios/RepositorioMateriasJson.cs

```

using System.Text.Json;

public class RepositorioMateriasJson : IRepositorioMaterias
{
    private readonly string _path;
    private readonly IRepositorioAlumnos _repoAlumnos;
    private readonly List<Materia> _materias;

    public RepositorioMateriasJson(string path, IRepositorioAlumnos repoAlumnos)
    {
        _path = path;
        _repoAlumnos = repoAlumnos;
        _materias = Cargar();
    }

    private List<Materia> Cargar()
    {
        if (!File.Exists(_path)) return new List<Materia>();
        string json = File.ReadAllText(_path);
        var dtos = JsonSerializer.Deserialize<List<MateriaDto>>(json) ?? new();
        return dtos.Select(dto =>
        {
            var materia = new Materia(dto.Codigo, dto.Nombre, dto.Cupo);
            foreach (int legajo in dto.LegajosInscriptos)
            {
                var alumno = _repoAlumnos.BuscarPorLegajo(legajo);
                if (alumno != null) materia.Inscribir(alumno);
            }
            return materia;
        }).ToList();
    }

    public void Guardar()
    {
        var dtos = _materias.Select(m => new MateriaDto
        {
            Codigo = m.Codigo, Nombre = m.Nombre, Cupo = m.Cupo,

```

```

        LegajosInscriptos = m.Inscriptos
            .OfType<Alumno>().Select(a => a.Legajo).ToList()
    }).ToList();
    string json = JsonSerializer.Serialize(dtos,
        new JsonSerializerOptions { WriteIndented = true });
    File.WriteAllText(_path, json);
}

public void    Agregar(Materia m)           => _materias.Add(m);
public Materia? BuscarPorId(string id)      => BuscarPorCodigo(id);
public Materia? BuscarPorCodigo(string c) =>
    _materias.FirstOrDefault(m => m.Codigo == c);
public List<Materia> ObtenerTodos()        => new(_materias);
}

```

Ejercicio 4.A: Implementar la persistencia JSON

1. Creá la carpeta DTOs/ con AlumnoDto.cs y MateriaDto.cs.
2. Agregá ObtenerNotas(), CargarNotas() y NombreEstrategia en Alumno.
3. Implementá EstrategiaFactory.cs.
4. Implementá RepositorioAlumnosJson y RepositorioMateriasJson.
5. Descomentá la rama "2" en Program.cs y probá la persistencia: agregá un alumno, salí del sistema y volvé a abrirlo. ¿Sigue estando?

Ejercicio 4.B — DESAFÍO: Guardado automático

Actualmente Guardar() se llama solo cuando el usuario elige "0. Salir".

Modificá el Controller para que llame a model.GuardarTodo() después de cada operación que modifica datos.

¿Qué ventaja tiene guardar después de cada cambio? ¿Qué desventaja podría tener en términos de rendimiento?

Paso 5 — El Controller con manejo de excepciones

Capturar errores específicos

Con las excepciones personalizadas del Paso 1, el Controller puede ahora capturar errores específicos y reaccionar de forma apropiada a cada uno. Observá el patrón **intentar** → **capturar** → **informar**: la lógica queda limpia y cada tipo de error tiene su propio mensaje.

EscolarController.cs — AgregarAlumno con manejo de errores

```
private void AgregarAlumno()
{
    try
    {
        string nombre = _view.PedirTexto("Nombre: ");
        string email = _view.PedirTexto("Email: ");
        int legajo = _view.PedirEntero("Legajo: ");

        _model.AgregarAlumno(new Alumno(nombre, email, legajo));
        _model.GuardarTodo();
        _view.MostrarMensaje("Alumno agregado correctamente.");
    }
    catch (LegajoYaExisteException ex)
    {
        _view.MostrarError(ex.Message);
    }
    catch (FormatException)
    {
        _view.MostrarError("El legajo debe ser un número entero.");
    }
}
```

EscolarController.cs — InscribirAlumno con manejo de errores

```
private void InscribirAlumno()
{
    try
    {
        int legajo = _view.PedirEntero("Legajo del alumno: ");
        string codigo = _view.PedirTexto("Código de materia: ");

        var alumno = _model.BuscarAlumno(legajo)
            ?? throw new EntidadNoEncontradaException("Alumno", legajo.ToString());

        var materia = _model.BuscarMateria(codigo)
            ?? throw new EntidadNoEncontradaException("Materia", codigo);

        materia.Inscribir(alumno);
        _model.GuardarTodo();
        _view.MostrarMensaje($"{alumno.Nombre} inscripto en {materia.Nombre}.");
    }
    catch (EntidadNoEncontradaException ex) { _view.MostrarError(ex.Message); }
    catch (SinCupoException ex) { _view.MostrarError(ex.Message); }
    catch (InvalidOperationException ex) { _view.MostrarError(ex.Message); }
}
```

Agregar MostrarError a la View

Agregá este método a **EscolarView** para distinguir visualmente los mensajes de error de los informativos. La View sigue siendo la única responsable de la presentación:

EscolarView.cs — nuevo método MostrarError

```
public void MostrarError(string mensaje)
{
    ConsoleColor prev = Console.ForegroundColor;
    Console.ForegroundColor = ConsoleColor.Red;
    Console.WriteLine($"  Error: {mensaje}");
    Console.ForegroundColor = prev;
}
```

Guardar al salir del sistema

En el método **Ejecutar()** del Controller, cuando el usuario elige "0. Salir", llamamos a **GuardarTodo()** para asegurar que los datos queden persistidos antes de cerrar:

EscolarController.cs – Ejecutar() actualizado

```
public void Ejecutar()
{
    bool continuar = true;
    while (continuar)
    {
        _view.MostrarMenu();
        string opcion = _view.PedirTexto("") ?? "";

        switch (opcion)
        {
            case "1": ListarPersonas(); break;
            case "2": AgregarAlumno(); break;
            case "3": AgregarDocente(); break;
            case "4": AgregarMateria(); break;
            case "5": InscribirAlumno(); break;
            case "6": VerReporteMateria(); break;
            case "7": VerPromedio(); break;
            case "8": CambiarEstrategia(); break;
            case "0":
                _model.GuardarTodo();
                _view.MostrarMensaje("Datos guardados. Hasta luego!");
                continuar = false;
                break;
            default:
                _view.MostrarError("Opcion invalida. Ingrese un numero del 0 al 8.");
                break;
        }
    }
}
```

Ejercicio 5.A: Controller completo con manejo de excepciones

1. Actualizá todos los métodos del Controller para que usen try/catch con las excepciones personalizadas.
2. Implementá el método AgregarMateria() que todavía no existe.
3. Asegurate de que GuardarTodo() se llama al salir del sistema.
4. Probá el flujo completo: opción 2 (JSON) → agregó datos → cerrá → reabrí → verificá que persisten.

Ejercicio 5.B — DESAFÍO: Validar email

Creá una excepción EmailInvalidoException que se lanza cuando el email no contiene "@".

Lanzala en el constructor de Persona antes de asignar el valor.

Capturala en el Controller con un mensaje claro al usuario.

Resumen: Estructura final del proyecto

Archivo	Carpeta	Descripción
Persona.cs	Model	Clase abstracta base
Alumno.cs	Model	Hereda Persona · ICalificable · IReportable · Strategy · persistencia
Docente.cs	Model	Hereda Persona · IReportable
Materia.cs	Model	IInscribible · IReportable · lanza SinCupoException
IInscribible / ICalificable / IReportable	Interfaces	Contratos de comportamiento
IEstrategiaPromedio.cs	Strategy	Interfaz del patrón Strategy
PromedioSimple / Ponderado / SinExtremos	Strategy	Implementaciones de estrategia
EstrategiaFactory.cs	Strategy	Reconstruye estrategias desde string (para JSON)
LegajoYaExisteException.cs	Excepciones	Error de legajo duplicado
SinCupoException.cs	Excepciones	Error de cupo agotado
EntidadNoEncontradaException.cs	Excepciones	Error de búsqueda fallida
IRepositorio.cs	Repositorios	Interfaz genérica del patrón Repository
IRepositorioAlumnos.cs	Repositorios	Contrato específico para alumnos
IRepositorioMaterias.cs	Repositorios	Contrato específico para materias
RepositorioAlumnosMemoria.cs	Repositorios	Implementación en memoria
RepositorioMateriasMemoria.cs	Repositorios	Implementación en memoria
RepositorioAlumnosJson.cs	Repositorios	Implementación con persistencia JSON
RepositorioMateriasJson.cs	Repositorios	Implementación con persistencia JSON
AlumnoDto.cs	DTOs	Datos de Alumno para serialización
MateriaDto.cs	DTOs	Datos de Materia para serialización
EscolarModel.cs	MVC	Coordina los repositorios
EscolarView.cs	MVC	Presentación en consola + MostrarError()
EscolarController.cs	MVC	Coordinación con manejo de excepciones
Program.cs	Entrada	Menú de configuración + ensamblado del MVC